

# 03 AI学习

## 一、RAG

### 1.1 RAG解释

RAG(Retrieval Augmented Generation)→想要解决的问题

| 看到一个很有意思的解释： |                                                     |                                         |
|--------------|-----------------------------------------------------|-----------------------------------------|
| 回合           | 你                                                   | AI大模型                                   |
| 1            | 你是一个程序员，此时你看着 <b>内部文档</b> 开发着程序，不出所料，你遇到了问题，于是你问了AI | AI从来没见过你这份文档，于是开始一本正经的胡说八道（ <b>幻觉</b> ） |
| 2            | 但你是个聪明的程序员，于是你想到了办法：把 <b>文档和问题</b> 一起发给大模型          | 这次，AI给出了正确的回答，问题似乎解决了                   |
| 3            | 但很快又出现了新的问题，你的文档越来越大，而答案，可能只存在于文档中的一小段话里            | AI看到整份文档找不到重点，回答容易跑偏                    |
| 4            | 聪明的你很快就想到，你不再把整个文档发过去，而是只发送 <b>问题相关的部分</b>          | 这就是 <b>RAG</b>                          |

### 1.2 Embedding模型

前面提到了RAG技术，只发送和问题有关联的部分，但是我们如何判断哪些有关联呢？于是，我们引入了Embedding模型

和大语言模型不同，它的输出是一个固定长度的数组（比如一个1536维的数组）

输入一段文字(1) → 【Embedding】 → 固定长度的数组(2)

输入一段文字(3) → 【Embedding】 → 固定长度的数组(4)

我们通过对比(2) 和 (4)两组数组之间的距离，从而判断文字(1) 和 (3)是否具有关联性，或者说关联性有多高

距离的判断：

可以将每条信息看作一个**信息点**，将问题也看作一个**问题点**

在一个1000多维度的空间中，通过对比**问题点和信息点之间的距离**，来判断和问题关联度较高的信息点

将这些关联度较高的信息点和问题点，一起发给AI模型，这样AI模型看到的就全是和问题强相关的信息了，产生幻觉和回答跑偏的概率就小了很多

## 1.3 文档的处理

### 1.3.1 Chunking 切块

文档太长怎么办？

我们可以在用户提问之前，先把文档进行处理：

- 1) 将文档切成很多的小片段（按字数、按段落、按句子等）
- 2) 将每个小片段都进行Embedding处理 → 变成长度一致的向量（数组）
- 3) 将原始片段和向量的对应关系保存起来（向量数据库）

### 1.3.2 Vector Database 向量数据库

对于传统的数据库，我们更多的使用场景是，通过一个精确的值查找对应的关系

而向量不同，我们需要输入一个问题向量，从向量数据库中查找与问题向量距离最近的几个向量

## 1.4 过程总结



